Queues and Stacks

COP 3502C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

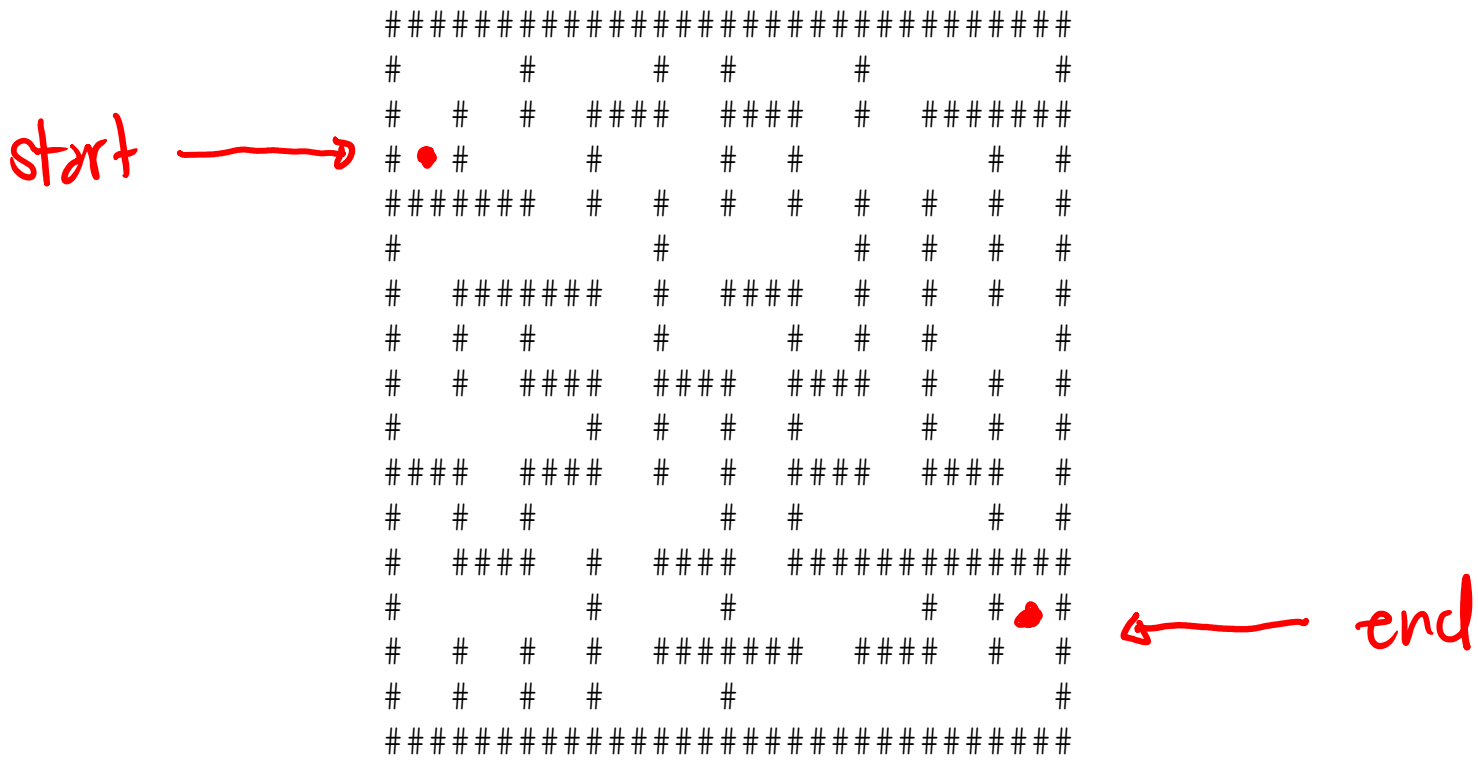
Scenario /1

- Write a program that can **parse** a mathematical expression and determine whether it is syntactically correct or not

- Further, it should be able to **evaluate** it

Scenario /2

- Write a program that can know how to exit a 2D maze



Recall

- Two **data structures**: arrays and linked lists

- **Concrete implementations** of how to store and organize data

Abstract Data Types (ADT)

- A conceptual model that is focused on **describing operations**

- **How it is implemented** is not the primary concern

Discussion /1

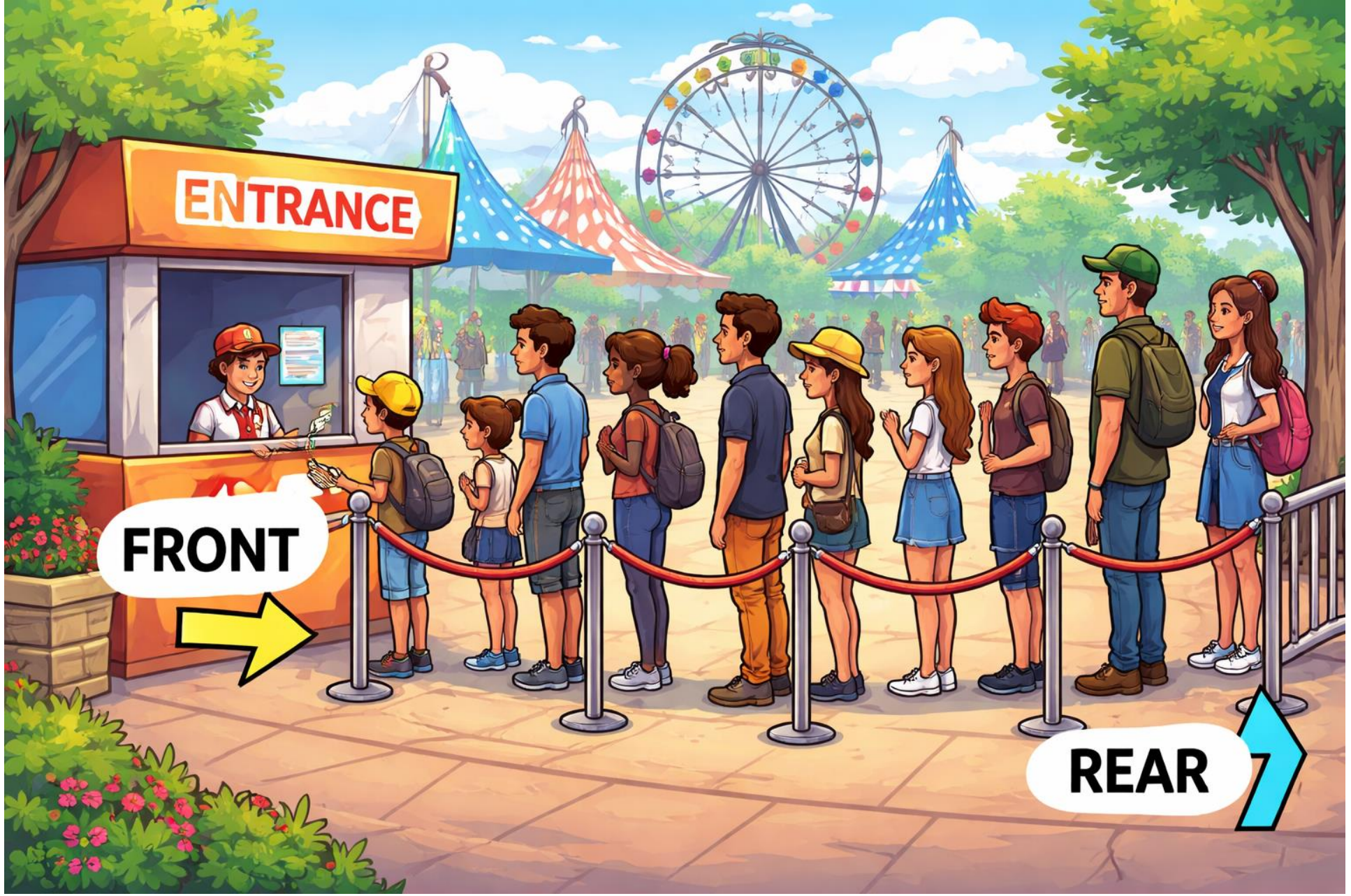
- We recall arrays and linked lists
- It is possible to **add** elements *anywhere* on the list
- The same is true for **removing** elements

Discussion /2

- Say, we **impose certain rules** on how we add or remove elements
- For example, we can add on one end then remove at the other end
- Or, we can only add or remove elements on one side

Queues and Stacks

- Examples of abstract data types
- **Think:** linear data structures that have restrictions
- Particularly on how it *grows* and *shrinks*



OpenAI. (2026). *A line in an amusement park* [AI-generated image]. ChatGPT. Retrieved from https://chat.openai.com/

OpenAI. (2026). *A pile of cafeteria trays* [AI-generated image]. ChatGPT. Retrieved from https://chat.openai.com/

Interlude

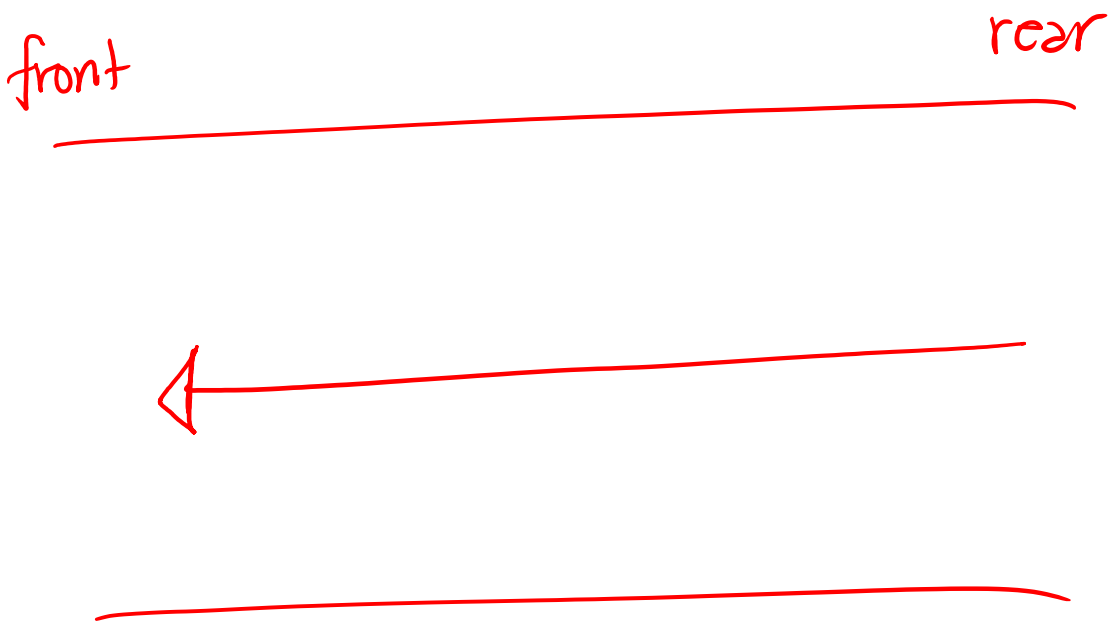
- For now, don’t worry about the actual implementation
- Focus on the **behavior** (i.e., operations)
- There’s a reason why it’s called an abstract data type

Queues

- Collection of items following **First In, First Out** (FIFO) principle
- What do we need to keep track of?
- Need to know the first and the last elements

Common Queue Operations /1

- `enqueue(val)`
- `dequeue()`
- `peek()` or `front()`
- `is_empty()`
- `is_full()`



Implementing Queues in C

Can be implemented using a **linked-list** or an **array**

Code Tracing

```
queue = queue_create();

for(int i = 1; i <= 5; i++) {
    queue_enqueue(queue, i);

    if( i % 2 == 0 ) printf("%d ", queue_dequeue(queue));
}

while( !queue_is_empty(queue) ) {
    if( queue_peek(queue) % 2 == 1 )
        printf("%d ", queue_dequeue(queue));
    else
        queue_dequeue(queue);
}

queue_destroy(queue);
```

Queue Applications

- **CPU Scheduling**
  - Determine which process gets to be processed by CPU

- **Queueing Theory**
  - Analyze and model scenarios (e.g., supermarkets or amusement parks)

Your Turn!

Determine the running time for the **worst-case scenario**

Queue Operations	**Linked List Implementation (`head` & `tail`)**	**Array Implementation (Naïve)**
`enqueue`		
`dequeue`		
`peek or front`		
`is_empty`		
`is_full`	N/A	

Notes

- The array implementation of the queue presented is not efficient
- Why?

Discussion

- Notice how we have been shifting the elements in the array?
- The array implementation presented was a **naïve** one
- Another approach is to use the concept of **circular arrays**

Circular Arrays

Assume that the capacity is 5

Enqueue `10, 20, 30, 40, 50`

Dequeue 2 times

Enqueue `60`

Your Turn!

Determine the running time for the **worst-case scenario**

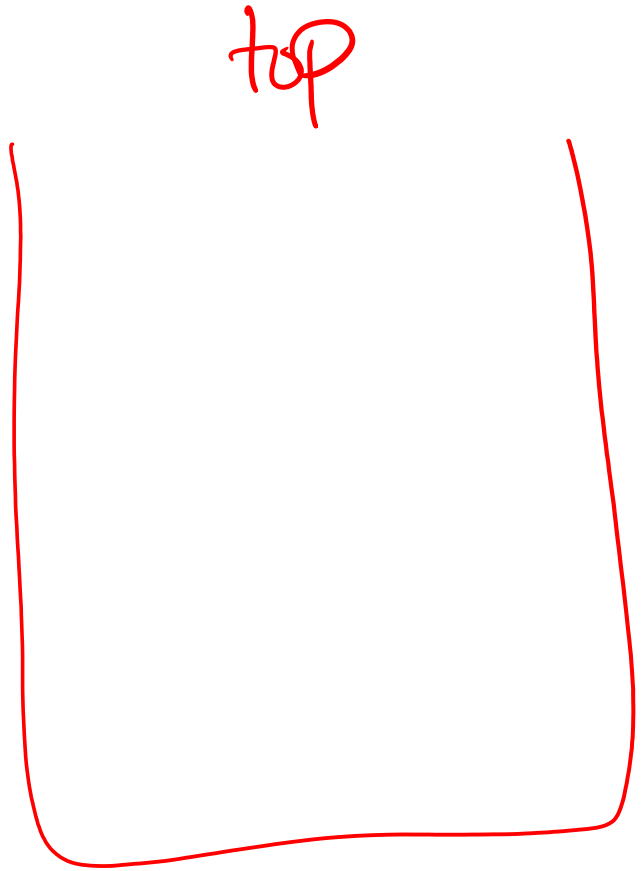
Queue Operations	**Linked List Implementation (`head` & `tail`)**	**Array Implementation (Circular)**
`enqueue`		
`dequeue`		
`peek or front`		
`is_empty`		
`is_full`	N/A	

Stacks

- Collection of elements following **Last In, First Out** (LIFO) principle
- What do we need to keep track of?
- Only need to know the **topmost** element of the stack

Common Stack Operations /1

- `push(val)`
- `pop()`
- `peek()` or `top()`
- `is_empty()`
- `is_full()`



Implementing Stacks in C

Can be implemented using a **linked-list** or an **array**

Code Tracing

```
stack = stack_create();

for(int i = 1; i <= 5; i++) {
    stack_push(stack, i);

    if( i % 2 == 0 ) printf("%d ", stack_pop(stack));
}

while( !stack_is_empty(stack) ) {
    if( stack_peek(stack) % 2 == 1 )
        printf("%d ", stack_pop(stack));
    else
        stack_pop(stack);
}

stack_destroy(stack);
```

Stack Applications /1

- **Infix and Postfix Notation***
  - Evaluate mathematical expressions

- **Backtracking**
  - Find candidate solutions by trial and error

Stack Applications /2

- **Call Stack**
  - Function call keeps track of where to go back afterward
  - Recall recursive functions

- **Converting Recursive Solutions**
  - Can be implemented iteratively when using a stack

Your Turn!

Determine the running time for the **worst-case scenario**

Stack Operations	**Linked List Implementation (`head` only)**	**Array Implementation**
`push`		
`pop`		
`peek or top`		
`is_empty`		
`is_full`	N/A	

Practice /1

Evaluate the following postfix notation (reverse Polish notation)

```
3 4 +
```

```
2 4 / 5 6 - *
```

Practice /2

Convert the following infix notation to postfix notation

```
3 + 4
```

```
( 2 / 4 ) * ( 5 – 6 )
```

Use a stack to temporarily store operators and parentheses while building the postfix output.

```
while there are more symbols to be read
    read the next symbol
    case:
        operand         --> output it.
        '('             --> push it on the stack.
        ')'             --> pop operators from the stack to the output
                            until a '(' is popped; do not output either
                            parenthesis.
        operator        --> pop higher- or equal-precedence operators
                            from the stack to the output; stop before
                            popping a lower-precedence operator or
                            a '('. Push the operator on the stack.
    end case
end while

pop the remaining operators from the stack to the output
```

Use a stack to store operands while evaluating the postfix expression.

```
while there are more symbols to be read
    read the next symbol
    case:
        operand    --> push it on the stack.
        operator   --> pop the top value as b
                       pop the next top value as a
                       compute a operator b
                       push the result back on the stack
    end case
end while

The final answer is the single value remaining on the stack
```

Notes

- Assuming we are only dealing with the operators: `+`, `-`, `*`, and `/`
- When you encounter an operator, peek and see if the top is not lower in precedence, if so, pop it then repeat
- Otherwise, proceed to the next symbol

Questions?